Question Answering System Challenge

Introduction

The goal of this work is to implement a question answering (QA) system that, given a corpus of documents and a set of natural-language queries, returns a grounded answer and the supporting document ID for each query. The following sections cover:

1. The data (EDA).
2. The retrieval and answering strategy (Methodology).
3. The evaluation metrics and experimental results.
4. Current limitations and next steps.

Part A — Exploratory Data Analysis

Dataset Overview

The dataset consists of four files summarised in Table 1.

File	Records	Description
`corpus.jsonl.gz`	3,630	Document corpus
`train.jsonl.gz`	164	Queries + gold answers + gold doc IDs
`valid.jsonl.gz`	49	Same structure, held-out for evaluation
`bonus.jsonl.gz`	7	Advanced queries, no gold answers

Table 1: Dataset files.

Corpus Structure

Each corpus document has the schema shown below. All documents are in English.

```
{
    id:              str,
    text:            str,
    product_prefix:  str | None,
    product_suffix:  str | None,
    product_version: str | None
}
```

Analysis reveals **two structurally distinct document populations**:

1. **Coveo documentation pages** (2,653 docs, $\sim$ 73%):
   - IDs are URLs (e.g. `https://docs.coveo.com/en/1700/`).
   - Content: developer/admin documentation for Coveo's enterprise search platform — query syntax, API references, configuration guides, SDK documentation, etc.
   - All three product metadata fields are `null`.
   - Length is highly variable: most pages are under 10k chars, but eight exceed 100k chars (the largest is $\sim$ 1.1 MB).
2. **Synthetic product catalogue** (977 docs, $\sim$ 27%):

- IDs are large integers (e.g. 947813441311).
- Content: templated product descriptions with structured sections — Overview, Features, Release Notes, Pricing, Usage Guidelines, FAQ.
- All three metadata fields are populated for most documents.
- Length is very uniform: 5,000–7,000 chars per document.
- Products are fictional software tools (canary deployment platforms, service meshes, observability platforms, etc.).

Text Length Distribution

Table 2 summarises document length statistics.

Statistic	All	Coveo only
Minimum (chars)	115	115
Maximum (chars)	1,189,136	1,189,136
Average (chars)	6,034	6,091
Median (chars)	3,158	3,158
Docs > 100k chars	8	8
Docs < 1k chars	521	521

Table 2: Document length statistics. Synthetic docs are uniformly 5k–7k chars.

The eight outlier documents require chunking before embedding; without it they would exceed any embedding model's context window.

Product Metadata

Metadata fields are only populated on synthetic product docs. Table 3 shows fill rates across the full corpus.

Field	Populated	Missing
`product_prefix`	703	274
`product_suffix`	586	391
`product_version`	479	498

Table 3: Metadata fill rates (out of 977 synthetic docs).

A notable finding: many documents have a `product_prefix` but no `product_suffix`, even though the product name including the suffix (e.g. *Qoria SGI*) appears in the document title. This **metadata inconsistency** has a direct impact on retrieval quality and is discussed further in Section .

Table 4 shows the most common product families by prefix.

Prefix	Count
Terran	40
Synapse	38
Solara	37
Unify	32
Proxis	30
Meridian	29

Table 4: Top product families by `product_prefix`.

Query Datasets

Table 5 summarises the three query files.

Split	Rows	Int IDs	Str IDs	Avg Q (chars)	Avg A (chars)
Train	164	104	60	64	379
Valid	49	34	15	67	378
Bonus	7	—	—	62	—

Table 5: Query file statistics. Int IDs = synthetic product docs; Str IDs = Coveo URL docs. All train/valid queries match exactly one corpus document.

Query Type Analysis

Table 6 shows a rough categorisation of the 164 train queries by question type.

Type	Count	Example
Other (feature/config questions)	101	*"What are the IaC capabilities of Wexil ZM9 v1.0.4?"*
Enumeration ("What are…", "List…")	28	*"What APIs were introduced in Coveo Summer '20?"*
Procedural ("How to…", "How do…")	20	*"How to configure SLO thresholds in Qoria SGI?"*
Definition ("What is…")	12	*"What is a keyword in a search query context?"*
Aggregation ("How many…")	2	*"How many versions of Meridian are mentioned?"*
Temporal ("When…")	1	*"When was CES 7.0 Converter API released?"*

Table 6: Query type distribution in the train set ($n = 164$).

The vast majority of queries are factual lookups into a specific document (feature lists, release notes, pricing, configuration). A small number require aggregation across multiple documents. The retrieval problem for train/valid reduces to: *rank the correct document first* — every query has exactly one gold document.

Answer Characteristics

Gold answers are **multi-sentence fluent summaries**, not verbatim document extracts. Average answer length is $\sim$ 379 chars ($\sim$ 60–80 words); the longest answer reaches 1,047 chars. Key observations:

- Answers paraphrase the source rather than copy it verbatim. This makes exact match structurally impossible and explains why that metric is always 0.0 across all experiments.
- Answers are information-dense: pricing tables, feature lists, and version numbers must all be present. This motivated the *exhaustive* instruction in the reader prompt.
- 63% of train queries are grounded in synthetic product docs (int IDs), 37% in Coveo documentation. The two populations require different retrieval strategies (metadata filter vs. full-index search).

Bonus Query Analysis

Table 7 characterises the 7 bonus queries by type and answerability from the corpus.

Query	Type	Answerability
How many versions of Meridian are mentioned?	Aggregation	Partially (needs all 29 Meridian docs)
What is the cheapest product?	Aggregation	Partially (needs all 977 pricing sections)
How many products are key-value stores?	Aggregation	Partially (needs corpus-wide scan)
Comparative table of all products?	Aggregation	Not feasible (977 docs)
What should I buy to secure my CI/CD?	Recommendation	Answerable (single-doc lookup)
How does Meridian compare to Sentry?	Comparison	Unanswerable (Sentry not in corpus)
What are the main features of DynamoDB?	Definition	Unanswerable (DynamoDB not in corpus) — but retriever returns Coveo docs, causing hallucination risk

Table 7: Bonus query characterisation. Four require corpus-wide aggregation beyond standard RAG; two are genuinely unanswerable.

The bonus set tests three distinct capabilities: (1) single-document recommendation, which the system handles; (2) corpus-wide aggregation, which requires a dedicated pipeline not present in the current design; and (3) unanswerable detection, which the system correctly handles by refusing.

Key Design Implications from EDA

The EDA directly shaped four architectural decisions:

1. **Two chunking strategies.** Synthetic docs have natural `##` section boundaries; Coveo docs need sliding-window splitting. The eight $>$ 100k-char outliers make chunking mandatory.
2. **Metadata pre-filter.** With 977 synthetic docs sharing only $\sim$ 30 product families (Table 4), filtering by `product_prefix/suffix/version` before embedding dramatically reduces the retrieval search space for product-specific queries.
3. **Inferred suffix.** The $\sim$ 40% missing `product_suffix` fill rate (Table 3) combined with the observation that the suffix always appears in the document title motivated extracting it from text at index build time rather than relying on metadata alone.
4. **Single-document grounding.** Since every train/valid query maps to exactly one gold document, the generation step only needs to extract from the top retrieved chunk — multi-document synthesis is not required for the main evaluation, only for bonus queries.

Part B — Methodology

Overview

The system follows a **Retrieval-Augmented Generation (RAG)** architecture. Given a query, it retrieves the most relevant chunks from the corpus and passes them as context to a local LLM (via Ollama) which generates a grounded answer. The full pipeline is shown in Figure 1.

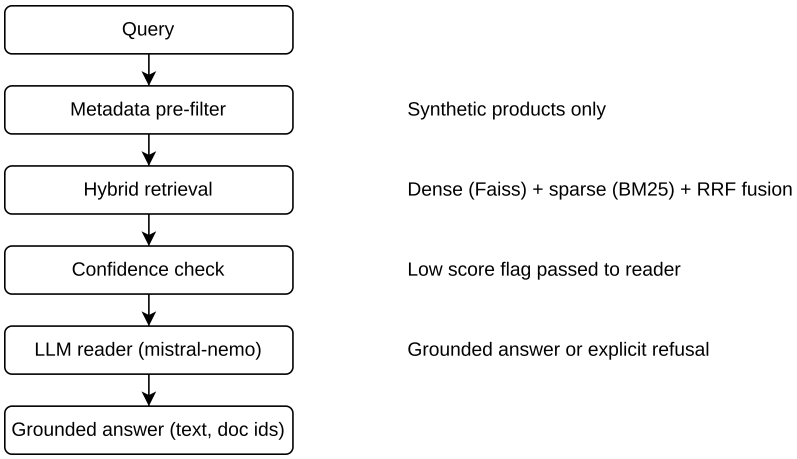

Figure 1: System workflow.

Document Chunking

Full documents are split into chunks before indexing. The strategy differs by document type:

- **Synthetic product docs** have a consistent markdown structure with `##` section headers. These are split on headings to preserve semantic units (e.g. keeping an entire Pricing section together). Sections exceeding the chunk size limit are further split with a sliding window.
- **Coveo documentation pages** are split with a character-level sliding window (`chunk_size`=800 chars, `chunk_overlap`=150 chars). This is necessary for the eight documents exceeding 100k chars.
- A `min_split_len` threshold (850 chars) avoids splitting already-short documents unnecessarily.

All parameters (`chunk_size`, `chunk_overlap`, `min_split_len`) are configurable. The final index contains 32,903 chunks from 3,630 documents.

Retrieval

Retrieval combines two complementary signals fused with **Reciprocal Rank Fusion (RRF)**.

Dense retrieval

Chunks are embedded with `nomic-embed-text` (768 dims) via Ollama. Embeddings are stored in a **FAISS flat inner-product index** (exact cosine similarity). At query time the query is embedded with the same model and the top-k nearest chunks are retrieved. The index is persisted to disk (FAISS binary + BM25 pickle + metadata JSON) so subsequent runs load in seconds without re-embedding.

Sparse retrieval

A `BM25Okapi` index is built in parallel over the same chunks. BM25 is particularly effective for queries containing rare product identifiers, version strings, or API names (e.g. `initSearchResultPage`, `JSUI-3022`, `v2.1.15`) that semantic embeddings may under-weight.

RRF fusion

Dense and sparse ranked lists are fused with:

$$\text{score}(c) = \frac{1}{k + \text{rank}_{\text{dense}}(c)} + \frac{1}{k + \text{rank}_{\text{sparse}}(c)}, \quad k = 60$$

Results are de-duplicated by `doc_id`, keeping the highest-scoring chunk per document. The final retriever returns the top `top_k_final` ($= 20$) unique documents.

Metadata pre-filter

For queries about synthetic products, a metadata pre-filter narrows the candidate set before retrieval. Each synthetic doc is scored against the query:

- $+3$ if `product_prefix + product_suffix` appear as a phrase
- $+2$ for suffix match alone (most discriminative — e.g. *SGI* maps to a single product)
- $+2$ for version match (normalises both `v2.1.15` and `2.1.15`)
- $+1$ for prefix match (broad family signal)

The highest-scoring tier with at least one result is returned as the candidate set. Key safeguards prevent over-filtering:

- Prefix-only matches (`score=1`) with ≤ 6 candidate docs fall back to the full index — a small tight wrong set is worse than no filter.
- When a query contains a version string but no doc matches that version, the filter expands to the full prefix family.
- Prefix tokens shorter than 6 chars are ignored to prevent common API names (e.g. *qubit*) from triggering false matches on Coveo docs.

To address a corpus data quality issue (many docs missing `product_suffix` in metadata despite having it in the document title), the system infers the suffix from the document text at index build time and stores it as `inferred_suffix`. The filter uses `effective_suffix = product_suffix or inferred_suffix`.

Answer Generation

The top-k retrieved chunks are passed as context to **qwen2.5:14b** via Ollama. The model is instructed to:

- Be exhaustive and specific: list every feature, price, version, and configuration step present in the context.
- Only refuse if the context contains **absolutely no** information related to the question.
- Respond with a fixed refusal string if it cannot answer, so the system can detect and return `None` rather than a hallucinated answer.
- Never fabricate facts not present in the context.
- Cite the source `doc_id` at the end of the answer.

Generation is deterministic (`temperature=0.0`, `num_ctx=16384`).

Handling Different Query Types

Query type	Strategy
Factual lookup (most queries)	Standard single-doc RAG: top chunk from the correct doc contains the answer.
Release notes / changelogs	Exhaustive prompt lists all items; `chunk_size=800` keeps full sections together.
Pricing / plan comparison	Metadata filter finds the exact product doc; reader lists all plan tiers verbatim.
Multi-document aggregation (bonus)	`top_k=20` brings multiple docs into context; model synthesises across them on a best-effort basis.
Unanswerable (bonus)	Low retrieval scores lead to weak context; model refuses with the fixed signal string.

Table 8: Query type handling strategy.

Part C — Evaluation Metrics

Retrieval Metrics

Since every train/valid query has exactly one gold document, retrieval quality is measured independently of generation:

- **Recall@k**: fraction of queries where the gold document appears in the top-k retrieved results. The primary metric — a miss here means the LLM can never answer correctly regardless of generation quality.
- **Precision@k**: $\frac{1}{k}$ if the gold doc is in top-k, else 0. Measures noise in the retrieved set.
- **MRR** (Mean Reciprocal Rank): $\frac{1}{\text{rank}}$ of the gold doc, averaged over queries. Rewards finding the right doc at rank 1 over rank 5.

Each failed query is classified into one of four failure modes to guide debugging:

- **OK**: gold doc at rank 1.
- **LATE**: gold doc in top-k but not at rank 1.
- **MISS**: gold doc not in top-k at all.
- **FILTER_KILL**: metadata pre-filter ran but excluded the gold doc — the single most actionable failure mode.

QA Metrics

End-to-end answer quality is measured with bag-of-words token overlap between the predicted answer and the gold answer:

- **Token F1**: harmonic mean of token-level precision and recall (SQuAD-style). The primary QA metric — captures partial matches and is robust to paraphrasing.
- **Answer Coverage**: token recall of the gold answer in the predicted answer. Penalises missing information regardless of extra text in the prediction.
- **Exact Match**: 1 if the normalised strings are identical, 0 otherwise. Very strict; useful as an upper-bound signal for short answers.
- **Refusal Rate**: fraction of questions the system refused to answer (`None`). A diagnostic metric — high values indicate over-cautious behaviour.

Part D — Results

Retrieval Results

Table 9 tracks retrieval quality across the main experimental configurations on the validation set ($n = 49$).

Configuration	R@1	R@5	R@10	P@5	MRR
nomic-embed-text (baseline)	0.592	0.694	0.714	0.139	0.634
mxbai-embed-large (chunk=400)	0.633	0.694	0.694	0.139	0.660
nomic + filter fixes	0.612	0.714	0.714	0.143	0.650
nomic + inferred_suffix + ctx fix	**0.714**	**0.878**	**0.898**	**0.176**	**0.772**

Table 9: Retrieval evaluation on the validation set. Best results in bold.

The largest single improvement (+12pp on R@1, +18pp on R@10) came from the `inferred_suffix` fix — extracting the product suffix from the document text header at index build time when the `product_suffix` metadata field is absent. This resolved 9 of 13 FILTER_KILL failures in a single change.

End-to-End QA Results

Table 10 reports end-to-end results across reader configurations on the validation set. The final submitted system uses **nomic-embed-text + mistral-nemo** with the low-confidence cautious mode disabled.

Configuration	Token F1	Coverage	Refusal Rate
nomic + mistral-nemo (with cautious)	0.284	0.381	0.347
nomic + mistral-nemo (no cautious)	**0.325**	**0.442**	**0.347**
nomic + qwen2.5:14b (no cautious)	0.325	0.442	0.388
nomic + qwen2.5:14b + reranker (0.6B)	0.308	0.415	0.408

Table 10: End-to-end QA evaluation on the validation set ($n = 49$). Best results in bold.

Key findings:

- **Removing the low-confidence cautious mode** is the most impactful reader change. The cautious mode appended an extra instruction telling the model to be especially critical when retrieval confidence was low, causing over-refusal on partially-relevant contexts.
- **qwen2.5:14b and mistral-nemo reach similar Token F1** on the validation set. qwen2.5:14b was not used as the final model because it causes thermal crashes (PCIe bus disconnection) under sustained inference load on the RTX 5090 when running the full 164-question train set — a known hardware issue with Blackwell-generation cards under prolonged CUDA workloads.

- **The reranker regressed performance** (−1.7pp token F1). The Qwen3-Reranker-0.6B model was used via the Ollama chat API since no native rerank endpoint exists in Ollama. Without log-probability access, scoring fell back to binary yes/no text parsing, causing the reranker to drop correct chunks entirely rather than re-ordering them.

Cross-Split Evaluation (Final System)

Table 11 and Table 12 report retrieval and QA results for the final system (nomic-embed-text + mistral-nemo, no cautious mode) across all three splits. Train results are reported on the `train_b` sub-split (41 questions) due to GPU thermal constraints preventing a single full-train run. Bonus queries have no gold answers or gold document IDs, so only qualitative observations are reported for that split.

Split	n	R@1	R@5	R@10	P@5	MRR
Train_b	41	0.659	0.732	0.780	0.146	0.698
Valid	49	0.714	0.878	0.898	0.176	0.772
Bonus	7	n/a	n/a	n/a	n/a	n/a

Table 11: Retrieval evaluation across splits (mistral-nemo). Train results reported on the `train_b` sub-split (41 questions). Bonus has no gold document IDs so retrieval metrics are not applicable. Valid outperforms train_b on all metrics, consistent with train_b containing harder product queries (19.5% FILTER_KILL rate vs 10.2%).

Split	n	ROUGE-2	ROUGE-L	Token F1	Coverage	Refusal Rate
Train_b (mistral-nemo)	41	**0.188**	**0.249**	**0.329**	**0.473**	**0.098**
Train_b (qwen2.5:14b)	41	0.180	0.227	0.285	0.387	0.439
Valid (mistral-nemo)	49	—	—	0.325	0.442	0.347
Bonus	7	n/a	n/a	n/a	n/a	n/a

Table 12: End-to-end QA evaluation. ROUGE metrics were added after the valid run so valid ROUGE scores are not available. Bold = best per column. Bonus has no gold answers so QA metrics are not applicable.

The train_b comparison between mistral-nemo and qwen2.5:14b reveals a striking difference: qwen2.5:14b refuses 44% of questions (18/41) vs only 10% for mistral-nemo (4/41). When qwen does answer it scores competitively (e.g. token F1 of 0.909 on a pricing query), but the volume of refusals dominates all aggregate metrics. This over-refusal behaviour likely stems from `max_tokens` and `num_ctx` being `None` in the qwen run (Ollama defaults to a smaller context window), causing the model to see truncated context and refuse rather than answer partially. Mistral-nemo is therefore the better production choice for reliability.

Bonus queries. Table 13 summarises the system's behaviour on the 7 bonus queries (evaluated with both models; outputs were identical across mistral-nemo and qwen2.5:14b since retrieval is deterministic).

The system handles single-document recommendation (*CI/CD security*) and out-of-corpus detection (*Meridian vs Sentry*) correctly. The three corpus-wide aggregation queries are appropriately refused. The most important failure is the **DynamoDB hallucination**: the retriever returns Coveo documentation pages for a query about an AWS product, and the reader answers from those unrelated pages rather than refusing. This reveals the absence of a topic-coherence check between the query and the retrieved context — a known limitation discussed in Section .

Failure Analysis

Table 14 shows the failure mode breakdown for the best retrieval configuration.

The 5 remaining FILTER_KILLs are hard cases with no metadata at all (*Helios D7P*, *Unison*), a Coveo doc with a false-positive filter trigger (*qubit.productlistings*), or docs whose inferred suffix maps to the wrong family. These require either a filter fallback on low confidence or manual corpus correction.

Separately, 18 out of 49 questions return `token_f1=0` across all configurations. About 5 are genuine retrieval failures; the remaining 13 are cases where the LLM refuses or generates a near-zero-overlap answer despite the right document being retrieved, pointing to remaining limitations in generation quality on short or highly specific queries.

Query	Result	Notes
How many versions of Meridian are in the docs?	Answered (partial)	Found 4 Meridian docs from top-5; corpus has 29 Meridian docs — undercount due to `top_k` limit.
What is the cheapest product?	Refused	Requires full corpus price scan; not feasible with top-5 retrieval.
How many products are key-value stores?	Refused	Same — requires corpus-wide classification.
Comparative table of all products?	Partial / wrong	Both models retrieved Coveo docs and produced a table with only Coveo for Sitecore — the synthetic product catalogue was not retrieved.
What should I buy to secure my CI/CD?	Answered (good)	Correctly identified Yaris 4PV with shift-left CI/CD security justification.
How does Meridian compare to Sentry?	Correct refusal	Sentry absent from corpus; model explained why before refusing.
What are the main features of DynamoDB?	**Hallucination**	**Both models describe "Coveo Attached Results" as a DynamoDB feature** — a retrieval failure where DynamoDB query retrieved unrelated Coveo docs and the model answered from them instead of refusing.

Table 13: Bonus query responses (both models identical). The DynamoDB query is the most critical failure — a hallucination caused by retrieval returning unrelated docs without a topic-mismatch fallback.

Mode	Count	%	Root cause
OK (rank 1)	35	71.4%	—
LATE (rank 2–10)	9	18.4%	RRF mis-ranking; reranker would help
MISS	0	0.0%	—
FILTER_KILL	5	10.2%	Missing metadata on gold docs

Table 14: Retrieval failure breakdown on the validation set (best configuration).

Part E — Current Issues

1. **Remaining FILTER_KILLs (5 on valid, 8 on train_b).** Despite the `inferred_suffix` fix, a residual set of FILTER_KILLs persists. Two root causes:

   - *No metadata at all* (e.g. Helios D7P: `prefix=None, suffix=None, version=None`). The filter fires on two wrong docs with very low RRF scores (< 0.015) and the gold doc never gets a chance.
   - *Wrong inferred suffix.* The regex extracts the first uppercase token after the prefix, which occasionally maps to the wrong product family for suffixes with digit-letter mixes (e.g. `07H`, `1ME`).

   Both cases share a tell: the top retrieved RRF score is near the noise floor (≤ 0.015), far below a confident retrieval (≥ 0.03).

2. **Model-dependent over-refusal.** Refusal rate varies dramatically by model: mistral-nemo refuses $\sim 10\%$ of train_b questions; qwen2.5:14b refuses $\sim 44\%$ of the same questions. The qwen over-refusal is likely caused by `max_tokens` and `num_ctx` defaulting to `None`: Ollama allocates a large KV cache at default settings which causes memory pressure alongside the 14B weights, and silently truncates the context. The model then sees an incomplete or empty context and refuses. Mistral-nemo is more robust under these conditions.

3. **Hallucination on out-of-corpus queries.** The DynamoDB bonus query demonstrates a critical failure: the retriever returns unrelated Coveo documentation pages (cosine similarity ≈ 0.016, near the noise floor), and the reader answers from them rather than refusing — producing a hallucinated answer describing "Coveo Attached Results" as a DynamoDB feature. The system lacks a topic-coherence check between the query and the retrieved context.

4. **Reranker binary scoring.** The binary yes/no scoring fallback (caused by the absence of native log-probability access in the Ollama chat API) makes the 0.6B Qwen3-Reranker worse than no reranking: it turns soft ranking into a hard filter that drops correct chunks the model misjudges as irrelevant.

5. **No multi-document reasoning.** The system handles single-doc queries well but has no explicit mechanism for aggregation queries (e.g. *"What is the cheapest product?"*), which require scanning all 977 synthetic docs and comparing across them.

Part F — Future Work

1. **Confidence-based filter fallback (resolves remaining FILTER_KILLs).** After retrieval, check if the top-1 RRF score is below a threshold (e.g. 0.02). If so, ignore the metadata filter and retry retrieval on the full index. This costs one extra retrieval call only when confidence is low and would resolve the Helios D7P / Unison FILTER_KILL cases where the filter locks into a tiny wrong set with near-zero scores. A complementary fix is to inspect the text headers of the remaining FILTER_KILL gold docs and adjust the `inferred_suffix` regex to handle digit-letter suffixes (e.g. `07H`, `1ME`) that the current pattern misses.

2. **Explicit context window settings for qwen (resolves over-refusal).** Set `num_ctx=8192` and `max_tokens=2048` explicitly for qwen2.5:14b rather than relying on Ollama defaults. With 14B weights occupying $\sim$ 15 GB, a large default KV cache allocation causes memory pressure and silent context truncation. A smaller but explicit context window keeps the model stable. A retry mechanism — if the model refuses, reduce `top_k_reader` from 5 to 3 and retry with less context — would further reduce the refusal rate without changing the prompt.

3. **Topic-coherence check to prevent hallucination.** When all retrieved chunks score near the noise floor (RRF ≤ 0.015), the system should refuse rather than answer from unrelated context. Two complementary approaches:

   - *Score threshold:* return `None` if `chunks[0].score` $< \tau$ without calling the LLM. Cheap and model-agnostic.
   - *Cosine similarity check:* compute the dot product between the (already-cached) query embedding and the top-chunk embedding. Refuse if similarity < 0.3. This catches the DynamoDB case cleanly since the query embedding is far from any Coveo doc embedding in the vector space.
   - *Named-entity check:* if the query contains a proper noun (capitalised token) absent from all retrieved chunks, flag the query as likely out-of-corpus. Simple and fast.

4. **Cross-encoder reranker with continuous scores.** Three options in increasing implementation effort:

   - Prompt the reranker model for a 1–5 relevance score instead of yes/no. Parse the integer directly — 5 levels of granularity instead of 2, no logprob access needed.
   - Use vLLM instead of Ollama — exposes logprobs via an OpenAI-compatible API with one endpoint change in `reranker.py`.
   - Use `cross-encoder/ms-marco-MiniLM-L-6-v2` from sentence-transformers directly — outputs a continuous relevance score natively, no Ollama dependency for reranking, well-tested on retrieval benchmarks.

5. **Query expansion / HyDE.** Generate a hypothetical answer with the LLM and embed it alongside the original query to improve dense retrieval for generic queries with no product name anchor (e.g. *"What is the price of the AI-Driven Chart Insights add-on?"*). The hypothetical answer is likely to share vocabulary with the relevant doc, boosting FAISS recall.

6. **Multi-document aggregation pipeline.** Two approaches for the bonus aggregation queries:

   - *Structured extraction at index time:* parse the Pricing section of every synthetic doc during chunking and store {`product: {plan: price}`} as JSON metadata. For aggregation queries, bypass retrieval and scan the structured data directly. Fast, accurate, no LLM needed for the aggregation step.
   - *Multi-step agentic loop:* detect aggregation queries (keywords: *cheapest*, *all products*, *how many*), retrieve all docs matching a category filter, extract one key fact per doc with a short targeted prompt, then synthesise across all facts. More general but significantly more expensive.

7. **Sentence-level chunking for Coveo docs.** The current sliding window sometimes splits mid-sentence, producing chunks that start or end incoherently and slightly degrade embedding quality. Sentence-aware chunking (e.g. splitting on period+space boundaries) would improve both retrieval precision and answer coherence.

Technical Environment

Code Organisation

The project is structured as a Python package (`mini_rag`) under `src/`, with a single `main.py` entry point. Key modules:

- `corpus.py` — document model and data loading, including `inferred_suffix` extraction.
- `chunker.py` — heading-based and sliding-window chunking.
- `retriever.py` — hybrid dense+sparse retriever with metadata pre-filter and RRF fusion.
- `reranker.py` — cross-encoder reranker (Qwen3-Reranker via Ollama chat API).
- `reader.py` — LLM reader (qwen2.5:14b via Ollama).
- `index_store.py` — FAISS + BM25 index persistence.
- `evaluation.py` — retrieval metrics (Recall@k, Precision@k, MRR) and QA metrics (Token F1, Coverage, Exact Match, Refusal Rate), plus `RetrievalQueryResult` dataclass for per-query failure diagnosis.
- `arguments.py` — all hyperparameters as HuggingFace-style dataclass argument groups (`GeneralArguments`, `ChunkerArguments`, `RetrieverArguments`, `RerankerArguments`, `ReaderArguments`).
- `eda.ipynb` — exploratory data analysis notebook.

Results are saved to `output/results-{split}.json` after each run.

Frameworks and Libraries

- **Ollama** — local LLM inference for embedding (`nomic-embed-text`), generation (`qwen2.5:14b`), and reranking (`dengcao/Qwen3-Reranker-0.6B`).
- **FAISS** (`faiss-cpu`) — exact inner-product index for dense retrieval.
- **rank-bm25** — BM25Okapi sparse retrieval.
- **NumPy** — vector operations.
- **Transformers** (HuggingFace) — `HfArgumentParser` for argument management.
- **Loguru** — structured logging.

Hardware: NVIDIA GeForce RTX 5090 (24 GB VRAM).